JOHANNES KEPLER
UNIVERSITY LINZ

Author
Jakob Waldhäusl
12015832

Submission
Institute for System Software (SSW)

Thesis Supervisor
Dipl.-Ing. **Thomas Schrott**, BSc.

August 2026

Performance Analysis of Data-oriented Programming in Rust



Bachelor's Thesis

to confer the academic degree of

Bachelor of Science

in the Bachelor's Program

Computer Science

JOHANNES KEPLER
UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Abstract

< Abstract in english >

Kurzfassung

< Abstract auf Deutsch >

Contents

Abstract ii

Kurzfassung iii

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 The Shift to DOP . . . 1
- 1.3 Objectives and Methodology . . . 1

2 Background 2
- 2.1 Data-Oriented Programming . . . 2
- 2.2 Caching and CPU-Architecture . . . 3
- 2.3 Data-Level Parallelism via SIMD . . . 3
- 2.4 Entity Component Systems . . . 4
- 2.5 Rust and Bevy engine . . . 5

3 Benchmark Design and Implementation 6
- 3.1 Architectures Evaluated . . . 6
- 3.2 Simulation . . . 6
- 3.3 Scenario 1: Compute-Bound . . . 6
- 3.4 Scenario 2: Memory-Bound . . . 6
- 3.5 Scenario 3: Allocation and Deallocation . . . 6
- 3.6 Execution and Configuration . . . 6

4 Evaluation 8
- 4.1 Methodology . . . 8
- 4.2 Scenario 1: Compute-Bound . . . 8
- 4.3 Scenario 2: Memory-Bound . . . 8
- 4.4 Scenario 3: Allocation and Deallocation . . . 8
- 4.5 Discussion . . . 8
- 4.6 Limitations . . . 8

5 Related Work 9

6 Future Work 10

7 Conclusion 11

Bibliography 12

List of Figures 13

List of Acronyms 14

Chapter 1

Introduction

1.1 Motivation

In the past decades, processor speeds have vastly increased due to higher cycle rates and higher core counts, while DRAM has not been able to keep up. This phenomenon is colloquially known as the "memory wall". The resulting problem is that execution speed may no longer be limited by processing power, but by the processor having to wait for data from the DRAM [4]. To address this gap, modern software development has to take into consideration the underlying hardware constraints.

1.2 The Shift to DOP

Object-oriented Programming (OOP) has enjoyed wide adoption and is usually taught long before Data-oriented Programming (DOP) in most curricula. However, OOP poses challenges when it comes to cache-optimization, for example, due to a high degree of coupling and inefficient memory layouts [6]. More modern approaches, such as DOP, aim to mitigate challenges such as these by decoupling logic and data. However, there are more mechanisms that DOP employs to improve cache efficiency and execution time, some of the more significant ones being Single Instruction, Multiple Data (SIMD) vectorization (Section 2.3) and use of the Entity Component System (ECS) software architecture design pattern (Section 2.4).

1.3 Objectives and Methodology

The aim of this thesis is to measure the difference in execution time between OOP and DOP approaches, as the exact difference is hardly ever quantified. The created benchmarks aim to reflect realistic use cases and lean towards the gaming industry, as there has been a significant push in recent years to adopt the ECS pattern. Rust will act as the programming language with the Bevy engine providing the framework. Rust provides memory safety without requiring a garbage collector [8]. The absence of garbage collection is especially desirable for benchmarking, as it eliminates nondeterministic collection pauses that would otherwise distort time measurements. The same holds for Bevy and especially the Bevy ECS, as it uses Rust's normal data types instead of requiring more complex features like lifetimes, traits or macros [2]. More intricate details of Rust and Bevy are explained and elaborated on further in section 2.5.

Chapter 2

Background

This section describes the relevant parts of DOP, which also include SIMD vectorization and the ECS design pattern, as they are integral to the nature of the systems under inspection. Furthermore, a brief overview of Rust and the Bevy engine is presented.

2.1 Data-Oriented Programming

The OOP paradigm undoubtedly has many useful features such as encapsulation, polymorphism and inheritance. These features enable software developers to think of problems in terms of objects and their interactions, with objects ideally being self-contained. Due to this, programmers are able to easily reuse and maintain them without having to fear side effects [1]. Nevertheless, objects encapsulating the entirety of their data and logic is a double-edged sword.

As an example, imagine a situation where you may want to update solely the location of some objects, with these objects having further attributes like color, weight etc. Due to encapsulation, the CPU has to load all the data of these objects into its cache, even though it should only have to load the location for its computation. This phenomenon is often called "cache pollution" [4] which poses a problem due to another phenomenon called the "memory wall".

Over the past decades, CPU clock speeds have increased exponentially. So did memory speed, however, at a far slower rate. According to Wulf et al. [13], in 1995 processing speed has been increasing by 80% per year with memory only achieving 7% per year. In the past 30 years, the memory wall has become one of the biggest bottlenecks in high-performance computing, due to the CPU often staying in idle while waiting for data to be fetched from main memory [7].

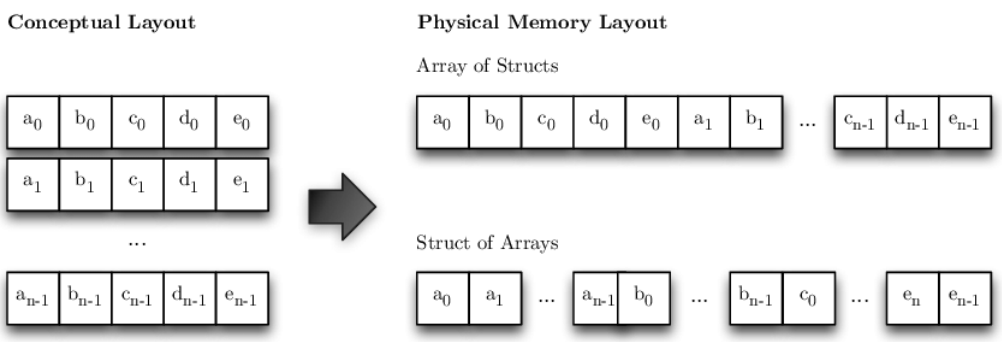


Figure 2.1: Memory Layout Visualization: AoS vs. SoA (taken from [10])

One of the most promising solutions to these problems is DOP, as it aims to decouple data from logic and thus reduce cache pollution and the CPU idling [5]. DOP improves CPU utilization by fundamentally changing how data is handled by moving away from the classical OOP memory layout called Array of Structures (AoS) and instead using a more optimized Structure of Arrays (SoA) layout [5].

In Figure 2.1, both memory layouts are compared. On the conceptual level, there are simply structs with data, e.g. objects with fields. The more conventional AoS layout stores these structs sequentially within an array, thus the name. In contrast, the SoA layout creates multiple arrays which contain only one type of field. As an example, one could want to manage 1000 `Person` objects which possess the fields `String name`, `float weight` and `Color hairColor`. The AoS layout would result in a `Person` Array with 1000 entries. Contrarily, the SoA layout would result in three arrays, one of type `String` containing the `name` values of all 1000 `Person` instances. Similarly, an array is created for `weight` and `hairColor`.

This is due to caches exploiting a principle called spatial locality. Drepper argues [4] that since neighboring data is likely to be used together, it should be loaded into the cache together. This is what the entire idea of SoA is based upon. If one wants to perform a read or write operation on only the `name` of these 1000 `Person` instances, the CPU can load much more relevant data than with the AoS layout, as the cache is not clogged by the irrelevant `weight` and `hairColor` data. Thus, while the SoA layout might appear confusing to programmers used to the AoS layout, it is necessary in order to properly achieve the performance gains granted by SIMD vectorization.

2.2 Caching and CPU-Architecture

2.3 Data-Level Parallelism via SIMD

Traditionally, to perform operations on data, the CPU has to load these values into a General Purpose Register (GPR). These usually have a fixed size of 32-bit or 64-bit, meaning an instruction can operate on two values of this size at once. For example, assuming a 32-bit environment, a scalar add instruction can add no more than two 32-bit values per cycle.

In order to overcome this limitation, modern CPUs contain specialized SIMD vector units. These units use much wider registers, such as the 512-bit registers found in the Intel MIC architecture, which allow the CPU to treat a single register as a vector of values. By using 512-bit registers, operations can be performed on up to 16 single-precision or 8 double-precision floating-point values in parallel [12]. On consumer grade CPUs, the AVX2 instruction set is widely adopted, using 256-bit registers.

This effectively parallelizes the computation and significantly increases throughput, under the condition that the data is laid out in memory as explained in Section 2.1. When looking at the memory layouts in Figure 2.1 it becomes clear why SIMD vectorization can only bring significant performance gains when the data is stored in a SoA layout. If the data was instead organized as in the AoS layout, SIMD units could not be fed with packed, homogenous data, causing the cache to be filled with irrelevant data that hinders the aspired parallel execution of operations. To achieve the outlined hardware efficiencies without the programmer having to manually manage memory, the ECS design pattern may be employed.

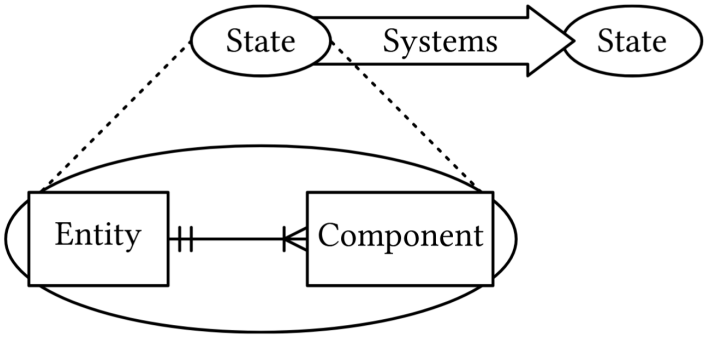


Figure 2.2: Visualization of the ECS pattern depicting (1) Entities, (2) Components and (3) Systems (taken from [11])

2.4 Entity Component Systems

As the presentation by Bilas [3] at the Game Developers Conference in 2002 shows, there has been a desperate need for the ECS pattern since at least the early 2000s. He explains that with increasing complexity of game systems, the resulting hardcoded class tree must undergo repeated refactoring and will naturally stray further and further from clean OOP principles. He noticed that when re-examining the problem of complex object schemes or structures, that he could also view it as a database and that in the end, only the data matters.

This data-centric perspective is the essence of the ECS pattern. It encourages clean separation of data and logic by employing a 3-level architecture as can be seen in Figure 2.2. (1) Entities can be viewed as an identifier value to which components may be attributed. (2) A Component is a domain-specific datatype and its value is comparable to the field value of an object. One entity may possess multiple components. (3) Lastly, a system is a domain-specific piece of logic or behavior. This system performs computations on entities and their components. Entities and Components represent a game state, with the Systems performing the transitions between states [11].

As an example, we revisit the `Person` objects from Section 2.1. We discussed that the SoA memory layout would greatly improve performance, but we did not explain how this layout would be achieved. Traditionally, one would define a `Person` class with fields and methods and then instantiate objects. To transform this into a SoA memory layout, the ECS pattern is applied as follows:

- Entities are created as unique identifiers.
- Components such as `name`, `weight` and `hairColor` are associated with these entities. In an archetype based ECS like Bevy, this ensures that all `weight` components are stored in a contiguous array, separate from `hairColor` [2].
- Lastly, Systems are defined to perform transformations by querying for entities that possess a certain set of components. For example, updating the `hairColor` component of all `Person` instances that possess the `name` and `hairColor` components. This allows the system to iterate over a tightly packed array of only relevant data, enabling the use of SIMD instructions.

2.5 Rust and Bevy engine

Rust is a systems programming language which provides performance comparable to C++ and memory safety through its unique ownership system. The compiler enforces strict rules on how data may be accessed and modified at compile-time. By doing so, Rust eliminates entire classes of bugs such as data races, and without requiring a garbage collector [8].

The Bevy engine expands upon this foundation, using an archetype-based ECS framework that prioritizes performance and modularity. Archetypes group entities with identical combinations of components, which enforces the SoA memory layout [2].

NOTE:
This section specifically will be expanded once the practical part is finished, as explaining Rust and Bevy in detail wouldn't make much sense and would likely be way too long. Focus will be on only the relevant parts for understanding the benchmarks once they are done.

Chapter 3

Benchmark Design and Implementation

3.1 Architectures Evaluated

- Transition from OOP vs DOP to the necissity of using certain data layouts to measure performance, from which The 4 architectures are gathered
- PC, AoS, SoA, Bevy

3.2 Simulation

- Describe planet calculation

3.3 Scenario 1: Compute-Bound

- Setup and Update Logic, SIMD

3.4 Scenario 2: Memory-Bound

- Setup and Update Logic, Cache Pollution / Memory Wall

3.5 Scenario 3: Allocation and Deallocation

- Setup and Update Logic, Drop vs Reuse

3.6 Execution and Configuration

All architectures and benchmarks are compiled into a single executable located at `../Benchmarks/target/`. An individual benchmark run is configured almost entirely through Command Line Interface (CLI) arguments, with the parsing of the commands being handled by the `clap` crate [9]. Table 3.1 lists the available arguments.

The program itself does not contain the functionality of parameter sweeping, i.e. varying parameter values within defined ranges. Instead, PowerShell and bash scripts are provided

Short	Long	Default	Description
`-b`	`--benchmark`	required	Scenario: `s1`, `s2`, `s3` or `reset`
`-p`	`--planets`	10 000	Number of planets created during setup
`-w`	`--warmups`	2500	Warmup iterations, results discarded
`-i`	`--iterations`	5000	Measured iterations per sample
`-s`	`--samples`	5	Number of samples per invocation
`-c`	`--chunksize`	100	Elements added and removed per iteration (S3)
`-r`	`--reuse`	`false`	Enables the reuse variant (S3)
`-d`	`--debug`	`false`	Prints a runtime summary to the console

Table 3.1: Command-line arguments for configuring the benchmark executable

which repeatedly execute the benchmark with varying values. This keeps the measured code free from any sweeping logic and makes every measurement reproducible from its arguments alone. In addition, the scripts recompile the benchmarks with varying payload sizes for Scenarios 2 and 3, as the payload size is a compile-time constant and thus must be defined before the execution of the benchmarks.

A single invocation executes all four architectures in sequence. For every architecture and for every sample, the corresponding benchmark function is called twice: once with the number of warumup iterations and once with the number of benchmark iterations. Calling the same function for both warmup- and benchmark iterations ensures that the cache is in a comparable state when the measurements begin. A typical invocation is shown below.

```
cargo run --profile o3 -- -b s3 -w 1000 -i 5000 -s 5 -c 500 -r -d
```

Measurement results are appended to a Comma-Separated Values (CSV) file per architecture and scenario. For example, the above invocation would append the measurements to the files `bevy.csv, soa.csv, etc.` located at `../Benchmarks/diagrams/s3/csv/`. Each line contains an entry for the respective architecture, planet count, number of iterations, chunk size, payload size in bytes as well as the elapsed time in nanoseconds. In order to delete the generated CSV files, the `reset` value for the `benchmark` argument may be used. This acts as a way to create a clean slate and prevent appending measurements to those of previous benchmarks, should the CSV files not have been moved yet.

Chapter 4

Evaluation

4.1 Methodology

- Benchmark Setup incl. Compiler Flags
- Hardware/System Description incl. CPU values

4.2 Scenario 1: Compute-Bound

- Results and Analysis
- SIMD Details

4.3 Scenario 2: Memory-Bound

- Results and Analysis
- "Anomalies", Cache

4.4 Scenario 3: Allocation and Deallocation

- Results and Analysis
- drop vs reuse

4.5 Discussion

- High Level Overview and Comparison of findings

4.6 Limitations

Chapter 5

Related Work

Chapter 6

Future Work

Chapter 7

Conclusion

Bibliography

[1] Tony Albrecht. 2008. Pitfalls of Object-oriented Programming. Technical report. https://www.slideshare.net/guestfe5b0d/pitfalls-of-object-oriented-programming-presentation.

[2] Bevy Project Contributors. 2026. *Bevy Quick Start Guide*. Bevy Engine. Retrieved 03/10/2026 from https://bevyengine.org/learn/book/introduction/.

[3] Scott Bilas. 2002. A data-driven game object system. In *Game Developers Conference Proceedings*. Retrieved 03/11/2026 from https://www.gamedevs.org/uploads/data-driven-game-object-system.pdf.

[4] Ulrich Drepper. 2007. What every programmer should know about memory. In volume 11. https://people.freebsd.org/~lstewart/articles/cpumemory.pdf.

[5] Richard Fabian. 2018. Data-oriented design. *framework*, 21. https://dataorientedesign.com/dodbook.pdf.

[6] Toni Härkönen. 2019. Advantages and Implementation of Entity-Component-Systems. *Tampere University, Tampere, Finland*, 6. https://urn.fi/URN:NBN:fi:tty-201905231735.

[7] John L. Hennessy and David A. Patterson. 2012. *Computer Architecture - A Quantitative Approach, 5th Edition*. Morgan Kaufmann. ISBN: 978-0-12-383872-8. https://www.academia.edu/download/54617908/CS6143_Computer_Architecture_-_A_Quantitative_Approach_5e.pdf.

[8] Steve Klabnik and Carol Nichols. 2023. *The Rust Programming Language*. No Starch Press. ISBN: 978-1-71-850444-8. https://doc.rust-lang.org/book/.

[9] Kevin Knapp. 2026. Clap 4.6.0. (2026). Retrieved 07/22/2026 from https://crates.io/crates/clap/4.6.0.

[10] Simon J. Pennycook, Simon D. Hammond, Steven A. Wright, J. A. Herdman, I. Miller, and Stephen A. Jarvis. 2013. An investigation of the performance portability of OpenCL. *J. Parallel Distributed Comput.*, 73, 11, 1439–1450. DOI: 10.1016/J.JPDC.2012.07.005. https://doi.org/10.1016/j.jpdc.2012.07.005.

[11] Patrick Redmond, Jonathan Castello, José Manuel Calderón Trilla, and Lindsey Kuper. 2025. Exploring the Theory and Practice of Concurrency in the Entity-Component-System Pattern. *Proc. ACM Program. Lang.*, 9, OOPSLA2, 1–28. DOI: 10.1145/3763050. https://doi.org/10.1145/3763050.

[12] Xinmin Tian, Hideki Saito, Serguei Preis, Eric N. Garcia, Sergey Kozhukhov, Matt Masten, Aleksei G. Cherkasov, and Nikolay Panchenko. 2013. Practical SIMD Vectorization Techniques for Intel® Xeon Phi Coprocessors. In *2013 IEEE International Symposium on Parallel & Distributed Processing, Workshops and Phd Forum, Cambridge, MA, USA, May 20-24, 2013*. IEEE, pp. 1149–1158. DOI: 10.1109/IPDPSW.2013.245. https://doi.org/10.1109/IPDPSW.2013.245.

[13] Wm. A. Wulf and Sally A. McKee. 1995. Hitting the memory wall: implications of the obvious. *SIGARCH Comput. Archit. News*, 23, 1, 20–24. ISSN: 0163-5964. DOI: 10.1145/216585.216588. https://doi.org/10.1145/216585.216588.

List of Figures

2.1 Memory Layout Visualization: AoS vs. SoA (taken from [10]) 2
2.2 Visualization of the ECS pattern depicting (1) Entities, (2) Components and (3) Systems (taken from [11]) 4

List of Acronyms

OOP Object-oriented Programming

DOP Data-oriented Programming

AoS Array of Structures

SoA Structure of Arrays

GPR General Purpose Register

SIMD Single Instruction, Multiple Data

ECS Entity Component System

CLI Command Line Interface

CSV Comma-Separated Values